

Functions

CS 101 Algorithmic Problem Solving

Fall 2024

1 What are Functions?

A python *function* is a block of statements that performs a specific task. The idea is to put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it over and over.

Some benefits of using functions are:

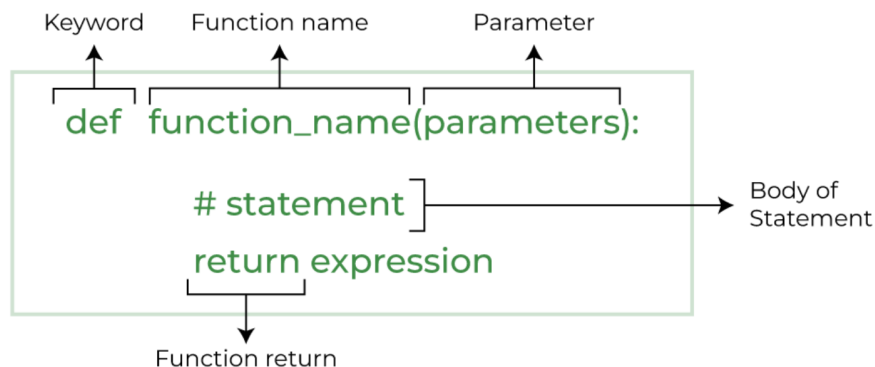
- Increase Code Readability
- Increase Code Reusability

2 Types of Functions

There are mainly two types of functions in python:

- Built-in library functions: These are standard functions in python that are available to use.
- User-defined function: We can create our own functions based on our requirements.

3 General format and examples



Parameters: are the variables that are defined when the function is declared.

Arguments: are the values passed inside the parenthesis of the function when it is called.

A function can have any number of arguments separated by a comma; it can even have no arguments. The arguments of a functions can be of many types:

- **Default Arguments:** A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument. The following example illustrates Default arguments.

```
def myFun(x, y=50):  
    print("x: ", x)  
    print("y: ", y)  
myFun(10)      #this will give x the value 10 and y will be 50, Function can  
               be called with one argument.  
myFun(10,20)   #though there is a default argument but when we explicitly give  
               the argument a value the value of y becomes 20 instead of  
               50.
```

- **Keyword/Named Arguments:** The idea is to allow the caller to specify the argument name with values so that the caller does not need to remember the order of parameters.

```
def student(firstname, lastname):  
    print(firstname, lastname)  
student(firstname='Geeks', lastname='Practice')  
student(lastname='Practice', firstname='Geeks')
```

In the above example, even though the order of arguments is different in the 2 class, the output will be the same because the arguments are used by name.

- **Positional Arguments:** If you forget the order of the arguments or change the positions of values, the output will be incorrect. To elaborate on this, an example is provided below:

```
def nameAge(name, age):  
    print("Hi, I am", name)  
    print("My age is ", age)  
# You will get correct output because argument is given in order  
nameAge("Suraj", 27)  
# You will get incorrect output because argument is not in order  
nameAge(27, "Suraj") #name and age will be swapped
```

Moreover, some functions are *fruitful* functions, which means they return some value when they are called. In contrast, *void* functions perform some action but do not return any value. All the functions provided above as examples are void functions as they do not return any value. An example of a fruitful function is provided below:

```
def CalAge(birth_year, current_year):  
    age = current_year - birth_year  
    return age  
print(CalAge(2002, 2023)) #21 will be printed, the value returned can be used  
                          directly to print.  
  
MyAge = CalAge(2002, 2023) #value returned from the fruitful function can be  
                           stored in a variable and used for other  
                           task.
```